

백엔드 및 데이터베이스(DB) 개발자이시군요! 그렇다면 UI의 화려함보다는 시스템 아키텍처, 데이터 모델링 역량, 대용량 트래픽 처리 경험, 비용 및 성능 최적화 같은 '보이지 않는 곳에서의 논리력'을 보여주는 것이 핵심입니다.

백엔드/DB 개발자의 강점을 극대화할 수 있도록 수정한 맞춤형 기획서입니다.

백엔드 & DB 개발자 포트폴리오 기획서

1. 개요 (Overview)

- **목적:** 백엔드 아키텍처 설계 역량 증명, 데이터베이스 최적화 경험 어필, 이직 및 프로젝트 수주
- **핵심 콘셉트: "신뢰성과 확장성(Reliability & Scalability)" * 톤앤매너:** 시스템 구조도와 수치화된 성과를 강조하는 차분하고 신뢰감 있는 디자인 (다크 모드 + 터키블루 또는 딥그린 포인트 컬러 추천)

2. 정보 구조 (Information Architecture)

- **Home:** 백엔드 개발자로서의 지향점 (예: "확장 가능한 시스템을 만드는...")
- **About Me:** 커리어 요약 및 기술적 관심사 (예: 분산 시스템, 쿼리 튜닝 등)
- **Tech Stacks:** 언어, 프레임워크, DB, DevOps/Infra 로 명확히 분류
- **System Architecture & Projects:** 백엔드의 핵심. 단순 화면 캡처가 아닌 구조도와 데이터 흐름 중심 배치
- **Contact:** GitHub, Tech Blog, LinkedIn 링크

3. 페이지별 상세 기획 (백엔드 맞춤형)

□ Section 1: Hero (첫화면)

- **콘텐츠 예시:** > "안정적인 데이터 흐름과 확장 가능한 아키텍처를 설계하는 백엔드/DB 개발자 000 입니다."
- **비주얼 요소:** 배경에 가벼운 가독성 좋은 코드 스니펫(API 엔드포인트나 데이터 모델 정의 등)이나 시스템 인프라 아이콘을 추상화한 그래픽 활용.

□ Section 2: Tech Stacks (기술 스택 분류)

백엔드 개발자는 기술의 깊이가 중요하므로 주력 기술과 서브 기술을 명확히 나눕니다.

- **Backend:** Java/Spring Boot, Node.js/Express, Python/FastAPI 등 (주력 언어 및 프레임워크)
- **Database & Cache:** PostgreSQL, MySQL, Oracle, Redis, MongoDB 등 (RDB/NoSQL 구분 및 캐싱 활용 능력 어필)
- **DevOps & Infra:** AWS (EC2, RDS, S3), Docker, Kubernetes, CI/CD (GitHub Actions, Jenkins)
- **Tools:** Git, Jira, Confluence

Section 3: Projects (포트폴리오의 핵심)

백엔드 프로젝트는 화면 캡처만으로는 무엇을 했는지 알기 어렵습니다. 반드시 **구조와 수치**가 보이아야 합니다.

- **UI 구성:** 프로젝트마다 [개요 - 시스템 아키텍처 구성도 - 기술적 도전과 해결(Troubleshooting)] 구조의 상세 페이지(또는 모달/접이식 메뉴) 제공.
- **필수 포함 내용:**
 1. **시스템 아키텍처 다이어그램:** 흐름도(유저 □□ LB □□ API 서버 □□ 캐시 □□ DB) 시각화.
 2. **데이터베이스 모델링 (ERD):** 핵심 테이블 구조나 관계 설계 이유 요약.
 3. **핵심 성과 (수치화):**
 - 예시: "Query 튜닝을 통해 느린 쿼리 수행 속도 45% 개선"
 - 예시: "Redis 캐싱 도입으로 API 응답 시간 200ms 에서 30ms 로 단축"
 - 예시: "DB 커넥션 풀 최적화를 통해 동시 접속자 처리량 2 배 증가"
 4. **Troubleshooting (가장 중요):** 어떤 장애나 병목이 있었고, 이를 어떤 논리로 해결했는지 작성 (예: 인덱스 부재 문제 해결, 데드락 개선 경험).

□□ Section 4: Tech Blog & Insights (선택 권장)

백엔드 개발자에게 기술 블로그는 강력한 무기입니다. 깊이 있게 고민한 흔적을 보여주세요.

- **구성:** 블로그, 티스토리, 미디엄 등 운영 중인 블로그의 최신 글이나 'DB 최적화 관련 글', '아키텍처 설계 고민' 등의 베스트 글 3~4 개를 카드로 연동.

4. 백엔드 개발자를 위한 구현 팁

- **Swagger/API 문서 연동:** 진행했던 프로젝트의 API 명세서(Swagger, Postman 리드미 등)로 바로 갈 수 있는 링크를 제공하면 프로페셔널해 보입니다.
- **GitHub 관리:** 프로젝트 링크를 클릭했을 때 대충 끝난 코드가 아니라, 코드 컨벤션이 지켜진 백엔드 코드와 상세한 README.md(아키텍처, 실행 방법, 테스트 커버리지 등)가 나오도록 빌드업해 두는 것이 홈페이지 자체보다 중요할 수 있습니다.